# Machine Learning and Deep Learning
# Lecture-12

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

# Overview

- Convolutional Neural Network (CNN)
- VGG, ResNet, MobileNets, and YOLO

# Convolutional Neural Networks (CNNs)

# Convolutional Neural Networks (CNN)

- CNNs were inspired by the study of the brain's visual cortex.
- They have been widely used for image recognition since the late 1980s (e.g., LeNet).
- Fully connected deep neural networks do not scale well to large images because they require too many parameters.
- For example, a 100×100 pixel image has 10,000 input units; connecting this to a first hidden layer of 1,000 neurons requires 10 million weights in just that layer.
- CNNs address this by using locally connected (partially connected) layers and weight sharing with convolutional filters.
- In CNNs, activations are arranged as 2D feature maps, which makes it easier to align neurons with their corresponding spatial inputs.

https://poloclub.github.io/cnn-explainer/

# CNN VS DNN

- CNNs
  - Layers are designed to process grid-like data (e.g., images).
  - Include convolutional layers that apply filters to detect edges, textures, and higher-level patterns.
  - Use far fewer parameters than fully connected networks on images → less overfitting risk.
  - Automatically learn spatial features from images.
  - Locally connected with weight sharing (convolutional filters).
- Applications: Image and video recognition, object detection, segmentation.
- DNNs
  - “Deep neural networks” include any neural network with multiple layers.
  - Typically consist of fully connected (dense) layers where each neuron connects to every neuron in the previous layer.
  - Have many more parameters on high-dimensional inputs → more prone to overfitting.
  - Often require more manual feature engineering for complex data like images.
  - Fully connected architecture (no built-in notion of spatial structure).
- Applications: Wide range of tasks (tabular data, time series, some text/image problems).

# CNN VS DNN

| Aspect | DNN (Fully Connected) | CNN |
|---|---|---|
| Input handling | Flattens input into 1D vectors, ignores spatial structure | Operates on grid-like data (e.g., images) and preserves spatial structure. |
| Layer type | Fully connected (dense) layers: each neuron connects to all previous neurons. | Convolutional + pooling layers with local connections and weight sharing. |
| Parameters | Many parameters, especially on high-dimensional inputs; prone to overfitting. | Far fewer parameters for images due to shared filters and local connectivity. |
| Feature learning | Often needs manual feature engineering | Automatically learns hierarchical visual features (edges → textures → objects). |
| Invariance | No built-in translation invariance | Convolutions provide translation invariance to learned patterns. |
| Typical applications | Tabular data, generic prediction tasks, some text/time-series | Image/video recognition, detection, segmentation, vision tasks. |

# Convolutional Neural Networks (CNN)

- Convolution is a mathematical operation that slides one function over another and measures the overlap via the integral of their pointwise product.
- It has deep connections to the Fourier and Laplace transforms (e.g., convolution in time $\leftrightarrow$ multiplication in the transform domain).
- In CNNs, the so-called “convolutional” layers actually use cross-correlation (very similar to convolution but without flipping the kernel).
- A CNN typically has three fundamental types of layers: convolutional layers, pooling layers, and fully connected layers.
- The convolutional layer is the most important building block of a CNN, responsible for feature extraction.
- During training, deep convolutional networks can require substantial GPU memory because convolutional layers produce many intermediate feature maps.

# Convolutional Neural Networks (CNN)

- Coloured image consists red, green, and blue with pixel intensity values from 0 to 255.
- A coloured image has a matrix shape of [height x width x channel].
- Side image of shape [10 x 10 x 3] indicating a 10 x 10 matrix with three channels.

- NN perform image processing on multi-channelled images.
  - Each channel represents a color.
  - Each pixel consists of three channels - RGB
- RGB image can be described as a wxhxn_c matrix, where w, h, and n_c denote width, height, and number of channels respectively.

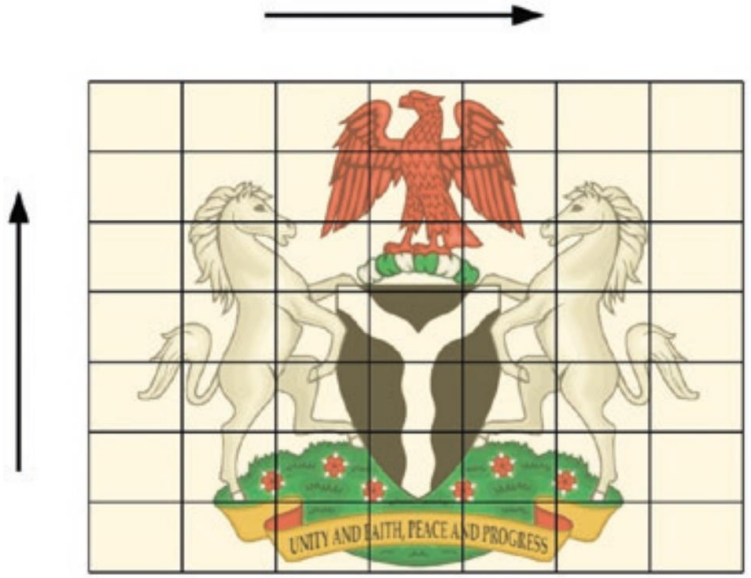
| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 250 | 250 | 255 | 246 | 249 | 251 | 245 | 251 | 250 | 250 |
| 249 | 255 | 246 | 206 | 118 | 97 | 183 | 241 | 255 | 250 |
| 253 | 253 | 218 | 60 | 8 | 6 | 28 | 203 | 254 | 254 |
| 255 | 242 | 226 | 89 | 37 | 45 | 89 | 214 | 230 | 253 |
| 254 | 231 | 208 | 235 | 122 | 112 | 235 | 213 | 217 | 255 |
| 254 | 238 | 203 | 253 | 139 | 111 | 254 | 204 | 228 | 251 |
| 255 | 234 | 229 | 196 | 114 | 101 | 155 | 230 | 233 | 255 |
| 253 | 247 | 254 | 55 | 93 | 132 | 0 | 215 | 252 | 253 |
| 251 | 253 | 236 | 144 | 74 | 74 | 121 | 221 | 255 | 252 |
| 250 | 255 | 249 | 242 | 218 | 209 | 239 | 246 | 253 | 249 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 250 | 250 | 255 | 236 | 216 | 209 | 231 | 255 | 252 | 250 |
| 251 | 254 | 234 | 161 | 78 | 52 | 120 | 223 | 255 | 250 |
| 253 | 255 | 173 | 43 | 5 | 8 | 4 | 148 | 255 | 255 |
| 255 | 249 | 123 | 0 | 50 | 60 | 2 | 101 | 217 | 255 |
| 255 | 230 | 94 | 54 | 53 | 25 | 119 | 113 | 179 | 255 |
| 255 | 226 | 130 | 214 | 49 | 2 | 150 | 136 | 208 | 255 |
| 255 | 216 | 218 | 205 | 109 | 94 | 147 | 216 | 210 | 255 |
| 254 | 244 | 237 | 47 | 87 | 122 | 0 | 178 | 243 | 255 |
| 251 | 254 | 197 | 69 | 61 | 60 | 51 | 165 | 255 | 252 |
| 248 | 255 | 250 | 203 | 156 | 137 | 188 | 249 | 253 | 249 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 250 | 250 | 253 | 224 | 103 | 97 | 202 | 252 | 251 | 252 |
| 248 | 255 | 212 | 6 | 8 | 21 | 4 | 183 | 253 | 248 |
| 253 | 254 | 54 | 35 | 118 | 119 | 64 | 31 | 239 | 255 |
| 253 | 205 | 2 | 73 | 103 | 103 | 83 | 0 | 175 | 252 |
| 255 | 165 | 0 | 0 | 58 | 67 | 0 | 0 | 132 | 253 |
| 253 | 150 | 2 | 23 | 74 | 83 | 65 | 27 | 119 | 255 |
| 253 | 150 | 50 | 236 | 77 | 42 | 255 | 109 | 106 | 255 |
| 255 | 229 | 33 | 120 | 53 | 37 | 127 | 35 | 202 | 254 |
| 251 | 255 | 138 | 2 | 68 | 83 | 0 | 106 | 255 | 252 |
| 249 | 253 | 252 | 148 | 33 | 29 | 122 | 254 | 253 | 249 |

# Convolutional Neural Networks (CNN)

- Image is depicted as a matrix of pixel intensity values ranging from 0 to 255.
- Grayscale consists of a single channel with 0 representing the black areas and 255 the white regions with the values in between for various shades of Gray.




**7 x 7 pixels**

2-D representation of an image

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 251 | 251 | 255 | 233 | 182 | 179 | 224 | 254 | 251 | 250 |
| 250 | 255 | 229 | 120 | 66 | 56 | 96 | 215 | 255 | 249 |
| 253 | 254 | 144 | 47 | 29 | 31 | 32 | 122 | 248 | 255 |
| 255 | 229 | 113 | 65 | 56 | 62 | 68 | 106 | 204 | 255 |
| 255 | 203 | 102 | 106 | 82 | 78 | 118 | 108 | 178 | 255 |
| 254 | 199 | 109 | 154 | 95 | 78 | 158 | 120 | 179 | 255 |
| 255 | 196 | 156 | 207 | 98 | 77 | 173 | 181 | 179 | 255 |
| 254 | 241 | 163 | 67 | 76 | 90 | 25 | 135 | 230 | 255 |
| 251 | 254 | 190 | 72 | 72 | 72 | 59 | 164 | 255 | 252 |
| 249 | 253 | 251 | 193 | 127 | 115 | 179 | 250 | 254 | 249 |

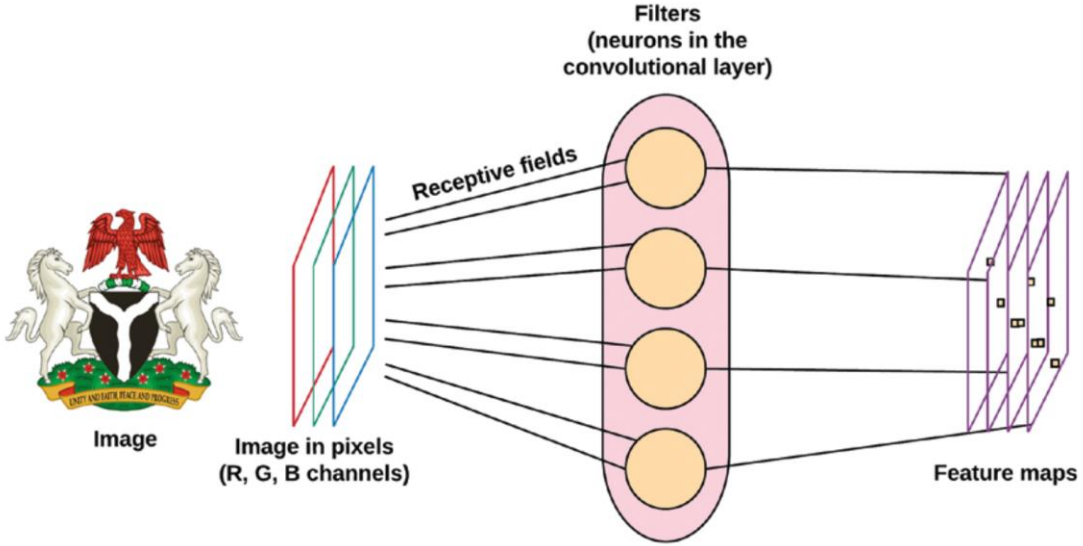
10 x 10 grayscale image with its matrix representation.

E. Bisong, Building Machine Learning and Deep Learning Models on Google Cloud Platform,

# CNN Components

- A convolutional layer is made up of filters (kernels) and feature maps.
- A filter (or kernel) is slid over the input image to capture specific features (e.g., edges or textures) in a process called convolution (implemented as cross-correlation in most CNN libraries).
- In this context, convolution applies a small weight matrix to local regions of the input to extract specific information from the image.
- Feature maps (activation maps) are the outputs produced by applying a filter across the input in a convolutional layer.

Receptive Field is an area of the input image that CNN neuron "sees"



# Design Convolutional layer

- Considerations to design convolutional layer:
  - **Filter size**: A neuron's weights can be viewed as a small image patch (its receptive field).
  - **Filters (kernels)**: Small matrices used to detect patterns in an image. Different filters learn to detect edges, textures, and shapes.
  - An $n \times n$ filter slides over the image and computes a dot product with each local patch (implemented as a convolution/cross-correlation).
  - **Stride**: The step size that determines how many pixels the filter moves between applications (default is 1). Larger strides reduce spatial resolution.
  - **Padding**: Zero padding adds a border of zeros around the image to control the spatial size of the output feature map.
  - Larger stride → smaller feature map (fewer details, faster computation).

**without zero padding**

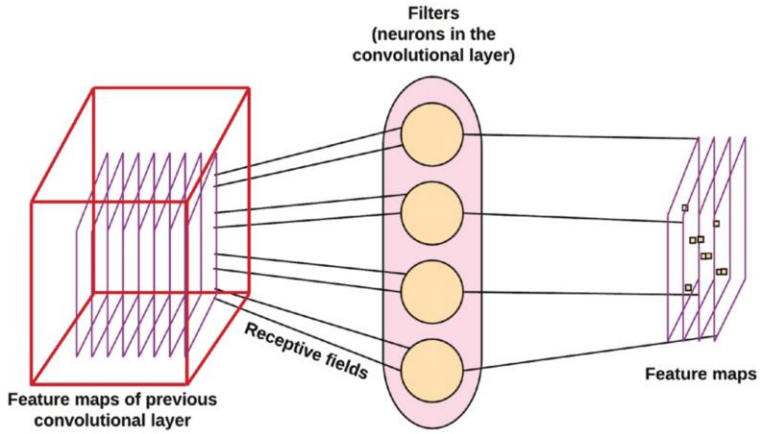
| | | | | | |
|---|---|---|---|---|---|
| 251 | 251 | 255 | 233 | 182 | 179 |
| 250 | 255 | 229 | 120 | 66 | 56 |
| 253 | 254 | 144 | 47 | 29 | 31 |
| 255 | 229 | 113 | 65 | 56 | 62 |
| 255 | 203 | 102 | 106 | 82 | 78 |
| 254 | 199 | 109 | 154 | 95 | 78 |

**with zero padding**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 251 | 251 | 255 | 233 | 182 | 179 | 0 |
| 0 | 250 | 255 | 229 | 120 | 66 | 56 | 0 |
| 0 | 253 | 254 | 144 | 47 | 29 | 31 | 0 |
| 0 | 255 | 229 | 113 | 65 | 56 | 62 | 0 |
| 0 | 255 | 203 | 102 | 106 | 82 | 78 | 0 |
| 0 | 254 | 199 | 109 | 154 | 95 | 78 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# CNN Components

- **Feature maps** are the output matrices produced by applying a filter in a convolutional layer.
  - They highlight patterns in the input image (e.g., horizontal edges, vertical edges, textures).
  - Using multiple filters yields multiple feature maps at the same layer.
  - In deeper CNNs, the inputs to a convolutional layer are the feature maps from the previous layer.
- **A pooling layer** summarizes image features learned in previous layers by down sampling the feature maps.
  - Pooling layers typically follow one or more convolutional layers.
  - The most common pooling operation is max pooling.
  - The goal of pooling is to reduce the spatial size of feature maps, lowering computation and introducing some translation invariance.



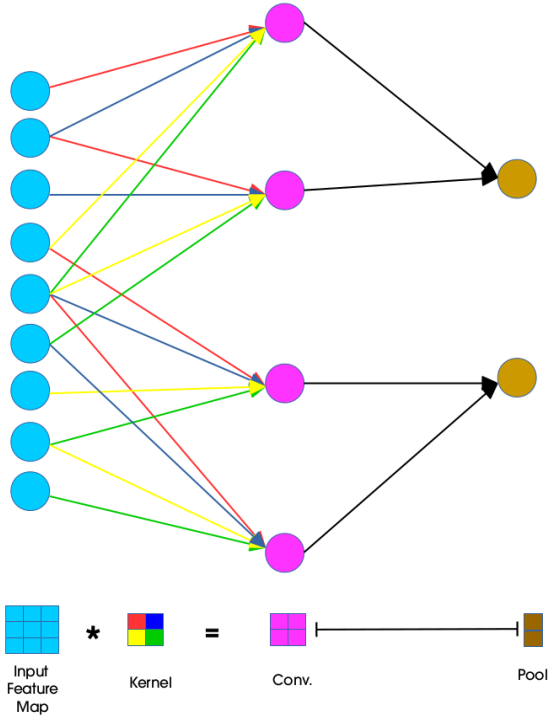

If we have a $32 \times 32$ RGB image and apply 32 filters, we get 32 feature maps.
Reason: Each filter processes the image to extract different features, resulting in a separate feature map for each filter.

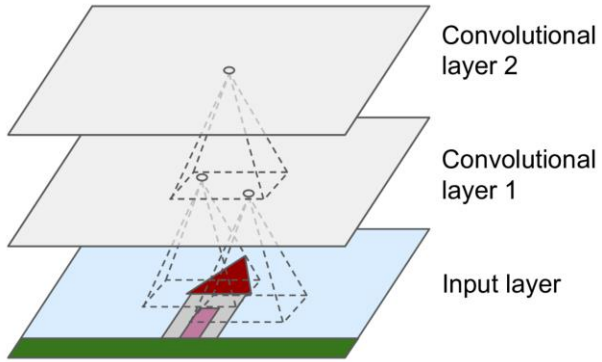
# Choosing "Correct" Kernel Size

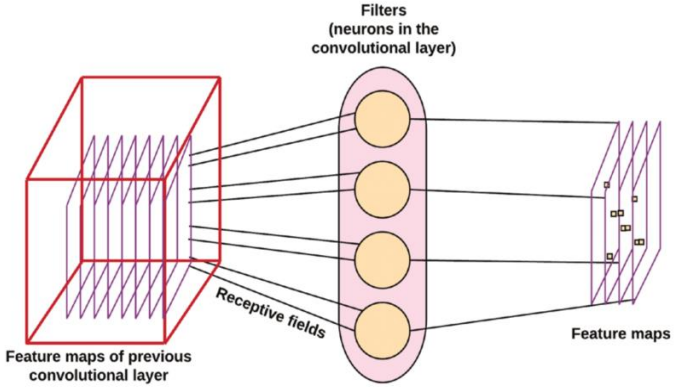
- Choosing the kernel size
  - Kernel size determines the receptive field of each convolutional operation.
  - Kernel size should be chosen based on the data characteristics and desired level of detail.
  - Fine-grained details → smaller kernels (capture local information).
  - Larger-scale patterns → larger kernels (capture more global structure).
  - Smaller input sizes typically work well with smaller kernels.
  - Larger input sizes can accommodate larger kernels, but they are more computationally expensive.
  - Stacking multiple layers with small kernels (e.g., several $3\times3$ layers) can approximate a larger receptive field while preserving more information.
  - Very large kernels may lead to over-smoothing and loss of fine-grained details.

# Convolutional Layer

- Convolutional layer: Neurons in the first convolutional layer are connected only to pixels within their local receptive fields, not to all input pixels.
- Each neuron in the second convolutional layer is connected only to a small region of neurons in the first layer (a local receptive field in the feature maps).
- This hierarchical local connectivity lets the network first learn small, low-level features in early layers and then combine them into larger, higher-level features in deeper layers.



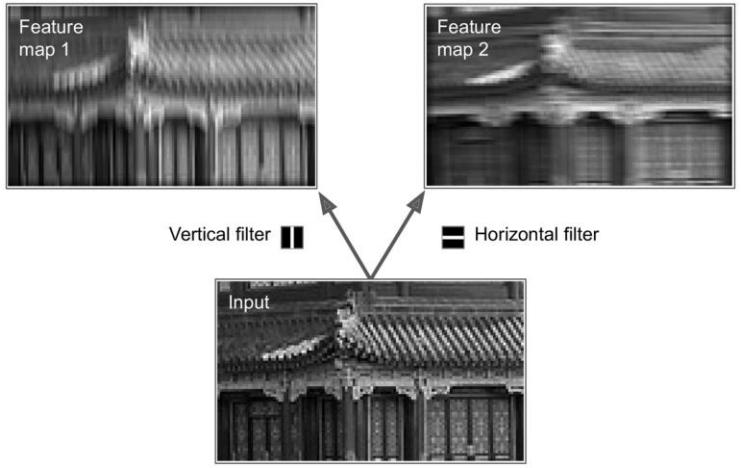




Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron

# Convolutional Layer

- A neuron's weights in a convolutional layer can be viewed as a small image patch corresponding to its receptive field.
- A feature map highlights regions in the input that strongly activate a particular filter (kernel).
- All neurons in a feature map share the same filter weights, which dramatically reduces the number of parameters compared to fully connected layers.
- Once a CNN has learned to recognize a pattern in one location, weight sharing allows it to recognize the same pattern anywhere in the input (translation equivariance).
- In contrast, a standard fully connected DNN without convolutions does not share weights across locations, so a pattern learned at one position does not automatically generalize to other positions.

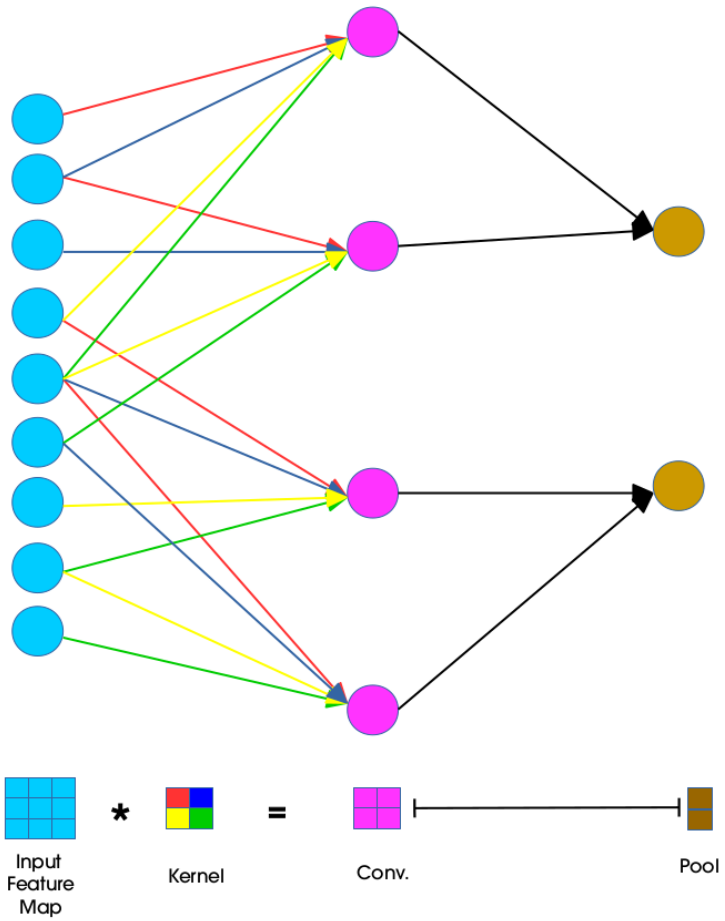


# CNN Components

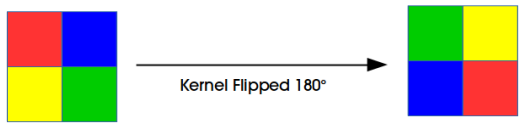
- A fully connected layer (dense layer) is a standard feedforward neural network or multilayer perceptron (MLP) layer.
- These layers typically use non-linear activation functions such as ReLU in hidden layers and softmax in the final classification layer (for probabilities).
- In many CNNs for classification, one or more fully connected layers appear at the end of the network to map extracted features to class scores.
- **CNN modeling**:
  - The first layer after the image input is typically a convolutional layer to extract local image features.
  - Pooling layers usually follow one or more convolutional layers to down sample feature maps

# Convolution Layer Training Process

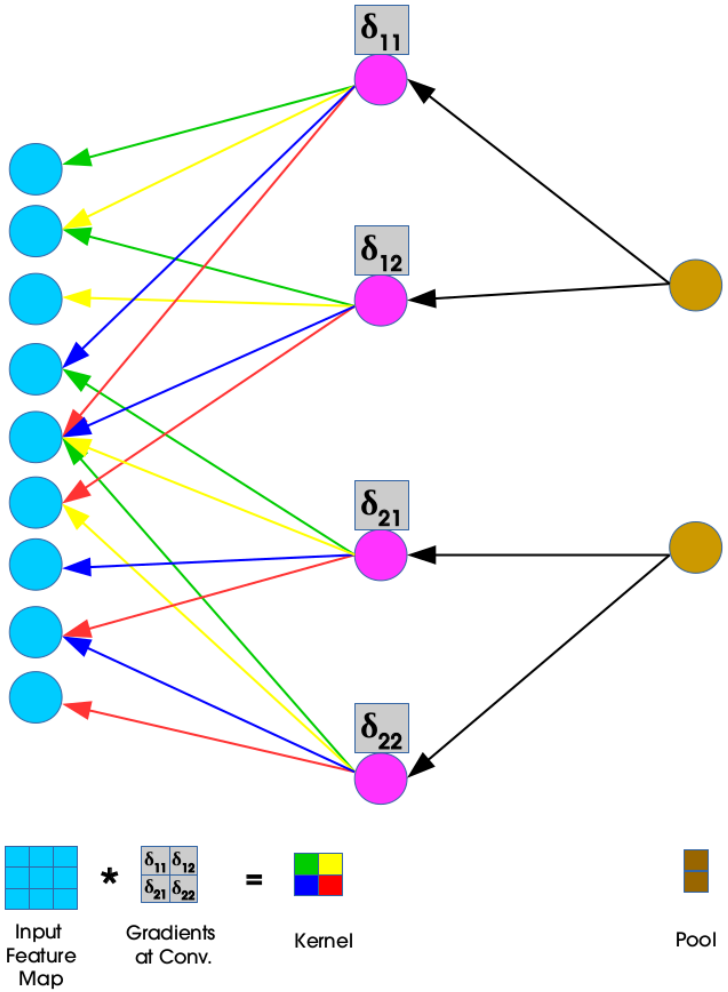
## Forward Propagation




No learning takes place on the pooling layers!!




## Backpropagation

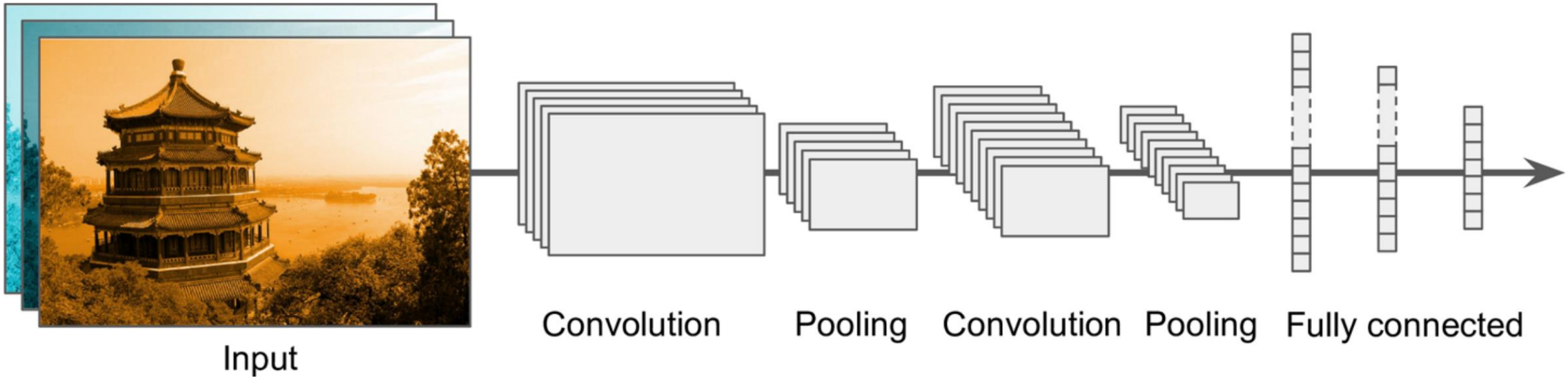



Gradients ==> $\delta_{11}, \delta_{12}, \delta_{21}, \delta_{22}$

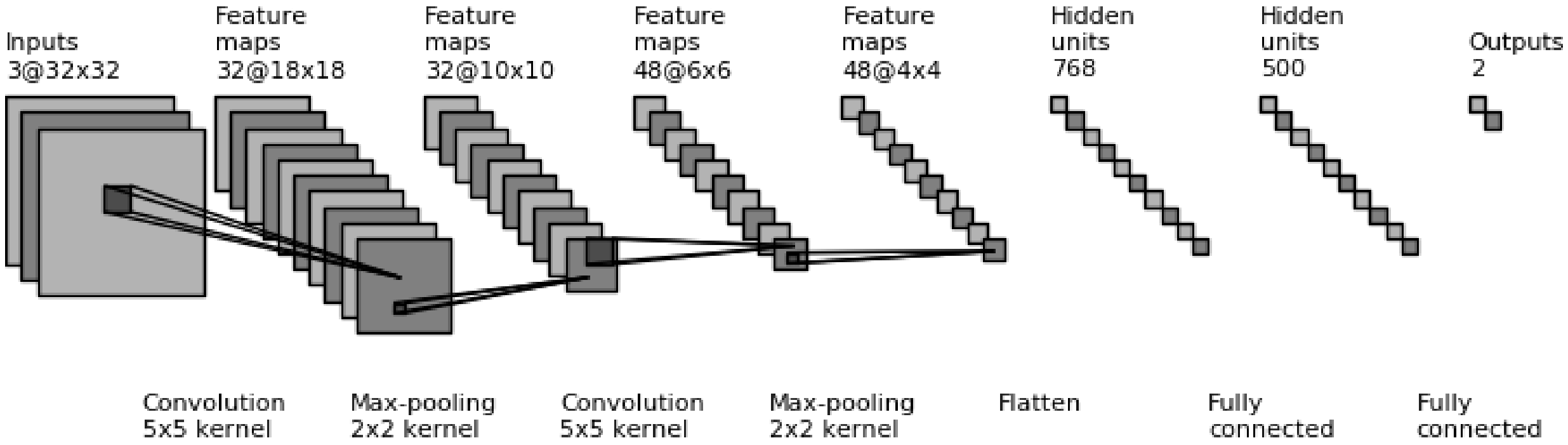
https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

# CNN Components

- In many classification CNNs, one or more fully connected (dense) layers are placed at the end, with the final layer producing class scores or probabilities.
- The last layer typically uses a softmax activation to output class membership probabilities; preceding dense layers often use ReLU or similar non-linear activations.
- CNNs may include one or more Dropout layers (often before or between dense layers) to reduce overfitting by randomly zeroing activations during training.



# CNN Example


Inputs
3@32x32
Feature
maps
32@18x18
Feature
maps
32@10x10
Feature
maps
48@6x6
Feature
maps
48@4x4
Hidden
units
768
Hidden
units
500
Outputs
2
Convolution
5x5 kernel
Max-pooling
2x2 kernel
Convolution
5x5 kernel
Max-pooling
2x2 kernel
Flatten
Fully
connected
Fully
connected

# Code Snippet

# Convolutional base using a common pattern: a stack of Conv2D and MaxPooling2D layers.
# CNN takes tensors of shape (image_height, image_width, color_channels), color_channels refers to (R,G,B) to support the format of CIFAR images.

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

| Layer Type | Filters | Kernel Size | Activation | |
|---|---|---|---|---|
| Conv2D | 32 | (3,3) | ReLU | This sample network processes 32x32 RGB images, extracting features through convolution and reducing spatial dimensions through pooling. |
| MaxPooling2D | - | (2,2) | - | |
| Conv2D | 64 | (3,3) | ReLU | |
| MaxPooling2D | - | (2,2) | - | |
| Conv2D | 64 | (3,3) | ReLU | |

Output is flattened and passed through dense layers for classification.

# Summary: CNN

- CNNs typically consist of convolutional layers, pooling layers, and fully connected (dense) layers.
- Convolutional layers are made up of filters (convolution kernels) and the resulting feature maps.
- Key considerations when designing a convolutional layer include:
  - Filter size: A neuron's weights can be viewed as a small image patch (its receptive field).
  - Stride: Determines how many pixel steps the filter takes when moving across the image (a stride of 1 is typical).
- Pooling layers usually follow one or more convolutional layers.
  - Their main goal is to reduce the spatial size of feature maps, lowering computation and adding some translation invariance.
- All neurons within a single feature map share the same parameters (the same filter weights).
- Training process:
  - Forward propagation: Pooling layers perform fixed operations; they do not have learnable parameters.
  - Backpropagation: The network's weights (e.g., in convolutional and dense layers) are updated based on the prediction error; deep convolutional networks can require substantial GPU memory due to many intermediate feature maps.

# VGG, ResNet, MobileNets, and YOLO Architectures

# VGG (Visual Geometry Group)

- **VGG Architecture**
  - Developed by the Visual Geometry Group at Oxford University (2014)
  - Deep convolutional neural networks designed for image classification
  - Known for simplicity and uniform architecture
- **Key Variants:**
  - **VGG-16:** 13 convolutional layers + 3 fully connected layers (138M parameters)
  - **VGG-19:** 16 convolutional layers + 3 fully connected layers (144M parameters)
- **Architecture Characteristics:**
  - Uses small 3×3 convolutional filters with stride 1 and padding 1
  - ReLU activation after each convolutional layer
  - Max pooling layers (2×2, stride 2) to reduce spatial dimensions
  - Simple, repetitive, and deep stacked structure
- **Strengths:**
  - Excellent feature extraction for transfer learning
  - Easy to understand and implement
  - Strong performance on ImageNet classification
- **Limitations:**
  - Very large number of parameters (memory intensive)
  - Computationally expensive to train

# ResNet (Residual Networks)

- **ResNet Architecture (**Winner ImageNet 2015 classification challenge**)**
  - Introduced by Microsoft Research, 2015
  - Enables training of very deep networks (50, 101, 152+ layers)
- **Core Innovation: Skip Connections (Residual Connections)**
  - Identity mappings that bypass one or more layers
  - Output: F(x) + x, where F(x) is the residual learned by the layers
  - Allows gradients to flow directly through the network during backpropagation
- **Why Skip Connections Work:**
  - **Mitigate vanishing gradients:** Provide a direct path for gradient flow
  - **Enable deep stacking:** Make training very deep networks feasible
  - **Learn residuals:** Layers learn only the difference (residual) between input and output
  - **Simplify identity mapping:** If a layer contributes little, weights can be near zero and input passes through unchanged
- **Architecture:**
  - Stacks residual blocks, each containing 2-3 convolutional layers with batch normalization and ReLU
  - Common variants: ResNet-34, ResNet-50, ResNet-101, ResNet-152
- **Impact:**
  - Revolutionized deep learning by enabling training of networks with 100+ layers
  - State-of-the-art performance on image classification, object detection, and segmentation

# MobileNet Architecture

- **Architecture**
  - Designed by Google for mobile and embedded devices (2017)
  - Optimized for efficiency: low computational cost and small model size
  - Enables real-time deep learning on resource-constrained devices
- **Core Innovation:** Replaces standard convolutions with a two-step process
- **1. Depthwise Convolution:**
  - Applies a single 3×3 filter to each input channel independently
  - Reduces computational complexity significantly
  - **2. Pointwise Convolution:**
  - Uses 1×1 convolutions to combine outputs across channels
  - Aggregates features from depthwise step into final representation
- **Benefits:**
  - **Parameter reduction:** 8-9× fewer parameters than standard convolutions
  - **Computational efficiency:** Significantly lower FLOPs (floating-point operations)
  - **Mobile-friendly:** Runs efficiently on smartphones and edge devices
- **Trade-offs:**
  - Slight accuracy reduction compared to full convolutions
  - Balanced speed-accuracy trade-off suitable for real-time applications
- **Variants:**
  - MobileNetV2 (inverted residuals, linear bottlenecks)
  - MobileNetV3 (neural architecture search, hardware-aware optimization)

# YOLO (You Only Look Once)

- **YOLO Object Detection**
  - Real-time object detection algorithm introduced by Joseph Redmon et al. (2015)
  - "You Only Look Once" refers to single forward pass through the network
  - Processes images at 45+ frames per second (real-time performance)

- **Core Concept: Single-Shot Detection**
  - Divides input image into an S×S grid (e.g., 7×7)
  - Each grid cell predicts bounding boxes and class probabilities simultaneously
  - If an object's center falls in a grid cell, that cell is responsible for detecting it

  **Evolution:**
  - YOLOv1 (2015) → YOLOv2-v3 → YOLOv4-v5 → YOLOv8-v11 (2024+)
  - Continuous improvements in speed, accuracy, and versatility

- **Architecture (YOLOv1):**
  - 24 convolutional layers + 4 max-pooling layers + 2 fully connected layers
  - Inspired by GoogLeNet architecture
  - Input image resized to 448×448 pixels

# YOLO (You Only Look Once)

- How YOLO Works:
  - Grid division: Image divided into grid cells
  - Bounding box prediction: Each cell predicts B bounding boxes with confidence scores
  - Class probability prediction: Each cell predicts class probabilities for detected objects
  - Non-Maximum Suppression (NMS): Filters overlapping detections

- Advantages:
  - Extremely fast (real-time detection on video streams)
  - End-to-end trainable single network
  - Reasons globally about the image (understands context)

# Summary

| Architecture | Year | Key Innovation | Primary Use Case | Trade-off |
|---|---|---|---|---|
| **VGG** | 2014 | Small 3×3 filters, deep stacking | Image classification, transfer learning | High parameter count |
| **ResNet** | 2015 | Skip connections (residual blocks) | Very deep networks (100+ layers) | Complexity in deeper variants |
| **MobileNet** | 2017 | Depthwise separable convolutions | Mobile/edge devices | Slight accuracy reduction |
| **YOLO** | 2015 | Single-shot detection (grid-based) | Real-time object detection | Speed-accuracy balance |